



كلية هندسة المعلوماتية

اكتشاف الأخطاء وإصلاحها

إعداد الطلاب :

أبي مازن الزحيلي أحمد طارق أبو عرب

أسماء المشرفين :

د. ميساء أبو قاسم م. وسام السحلي



College of engineering

Bug detection and fixing

Students preparation:

Obai Alzohili

Ahmad Aboarab

Names of supervisors:

E.wesam alsohle

D.Mesaa abo kassem

المخلص :

يهدف هذا المشروع إلى تصميم وتطوير نظام ذكي قائم على تقنيات الذكاء الاصطناعي والتعلم العميق لاكتشاف الأخطاء البرمجية (Software Bugs) وإصلاحها تلقائياً، بهدف تحسين جودة الشيفرات البرمجية وتقليل الوقت والجهد المبذول في عمليات الصيانة البرمجية. يعتمد النظام بشكل أساسي على نماذج حديثة مبنية على بنية ال-Transformer ، لما تتمتع به من قدرة عالية على فهم السياق البرمجي وتحليل البنية اللغوية والمنطقية للكود.

تم في هذا المشروع تدريب نموذج CodeT5 باستخدام مجموعات بيانات متخصصة تحتوي على أكواد برمجية خاطئة وأخرى صحيحة، مثل مجموعة بيانات PyresBugs وغيرها من البيانات المعتمدة في الأبحاث الحديثة، مما أتاح للنموذج تعلم أنماط الأخطاء الشائعة وآليات إصلاحها. شملت مرحلة الإعداد معالجة البيانات وتنظيمها، وتوحيد تمثيل الشيفرات، وتقسيمها إلى مجموعات تدريب وتقييم، مع ضبط معايير التدريب مثل معدل التعلم، عدد العصور التدريبية، وحجم الدفعات لضمان استقرار النموذج وتحقيق أفضل أداء ممكن.

إضافة إلى ذلك، تم الاستفادة من نموذج لغوي متقدم هو Gemini 2.5 Flash/GPT4-o-(mini) عبر واجهة برمجية (API Call) ، بهدف مقارنة أدائه مع النماذج المدربة محلياً. وقد أتاح هذا النهج تقييم فعالية النماذج اللغوية الكبيرة في اكتشاف الأخطاء البرمجية واقتراح تصحيحات دقيقة دون الحاجة إلى تدريب مخصص، اعتماداً على قدراتها المتقدمة في الاستدلال البرمجي وفهم السياق العميق للكود.

تم تقييم أداء النظام باستخدام مجموعة من المعايير المعتمدة في مجال تصحيح الأكواد، مثل CodeBLEU و Pass@k ، إضافة إلى التقييم العملي عبر تنفيذ اختبارات للتحقق من صحة الكود بعد الإصلاح. أظهرت النتائج أن نموذج CodeT5 حقق أداءً جيداً مقارنة بالأساليب التقليدية، في حين تفوق نموذج (Gemini 2.5 Flash/GPT4-o-mini) بشكل ملحوظ من حيث جودة التصحيحات المقترحة ومعدل نجاح الاختبارات بعد الإصلاح، مما يعكس قدرته العالية على التعامل مع السيناريوهات البرمجية المعقدة.

تؤكد نتائج هذا المشروع أهمية توظيف نماذج الذكاء الاصطناعي الحديثة، وبالأخص نماذج Transformer والنماذج اللغوية الكبيرة، في مجال اكتشاف وإصلاح الأخطاء البرمجية. كما يبرز المشروع إمكانية دمج النماذج المدربة محلياً مع نماذج مقدمة كخدمة عبر واجهات برمجية لبناء أنظمة أكثر كفاءة ومرونة، بما يساهم في تحسين موثوقية البرمجيات ودعم المطورين في بيئات التطوير الحديثة. ويمكن مستقبلاً تحسين النظام من خلال توسيع نطاق البيانات، دعم لغات برمجية إضافية، أو دمج آليات تحقق أعمق لضمان سلامة التصحيحات المقترحة.

Abstract :

This project aims to design and develop an intelligent system based on artificial intelligence and deep learning techniques for the automatic detection and repair of software bugs, with the objective of improving code quality and reducing the time and effort required for software maintenance. The proposed system primarily relies on modern models built on the Transformer architecture, which demonstrate a strong capability to understand programming context and analyze both the linguistic and logical structure of source code.

In this work, the CodeT5 model was trained using specialized datasets containing buggy and fixed code, such as the PyresBugs dataset and other widely adopted research benchmarks. This enabled the model to learn common bug patterns and corresponding repair strategies. The data preparation phase included code normalization, structured representation, and splitting into training and evaluation sets. Training hyperparameters—such as learning rate, number of epochs, and batch size—were carefully tuned to ensure training stability and optimal performance.

In addition to locally trained models, advanced large language models—namely Gemini 2.5 Flash and GPT-4o-mini—were utilized through API-based inference to compare their performance with the trained CodeT5 model. This approach allowed for an objective evaluation of the effectiveness of large language models in software bug detection and repair without task-specific fine-tuning, leveraging their advanced reasoning capabilities and deep contextual understanding of code.

The system was evaluated using widely adopted metrics in automatic program repair, including CodeBLEU and Pass@k, along with practical validation through automated test execution to verify code correctness after repair. Experimental results show that while CodeT5 achieved strong performance compared to traditional approaches, Gemini 2.5 Flash and GPT-4o-mini significantly outperformed locally trained models in terms of repair quality and post-fix test success rates, demonstrating superior robustness when handling complex programming scenarios.

The findings of this study highlight the effectiveness of modern Transformer-based models and large language models in advancing automatic software bug detection and repair. Furthermore, the project demonstrates the feasibility of integrating locally trained models with API-based large language models to build efficient and flexible repair systems, ultimately enhancing software reliability and supporting developers in modern development environments. Future work may focus on expanding dataset diversity, supporting additional programming languages, and incorporating deeper verification mechanisms to further ensure the correctness of generated fixes.

الفهرس

4	المُلخص :
8	قائمة بأهم المصطلحات :
8	مصطلحات البيانات وهيئتها
8	تقسيم ومعالجة البيانات
9	النماذج وخوارزميات الذكاء الاصطناعي
9	التدريب والتقييم
11	الفصل الأول :
11	التعريف بالبحث
12	المقدمة:
12	1.1 التمهيد :
13	2.1 مشكلة البحث :
13	3.1 النتائج و المساهمات :
14	4.1 هيكلية البحث :
15	الفصل الثاني:
15	الدراسات المرجعية
16	2.1 دراسات في مجال اكتشاف الأخطاء البرمجية وإصلاحها باستخدام نماذج الذكاء الاصطناعي
18	الفصل الثالث:
18	تحليل النتائج والمنهجية
19	3.1 التحليل التقني للنماذج المعتمدة على Transformer لتصحيح الأخطاء البرمجية
19	3.1.1 مقدمة حول استخدام نماذج Transformer في تصحيح الأكواد
19	3.1.2 البنية العامة لنموذج Transformer المستخدم
20	3.1.3 تقنيات التكيف منخفض المعلمات (LoRA) و (QLoRA)
21	3.1.4 آلية التدريب والإعداد الموحد للنماذج
21	3.1.5 معايير التقييم المعتمدة
22	3.1.6 تحليل النتائج التجريبية
23	3.2 التحديات التقنية
23	3.3 الخلاصة
24	الفصل الرابع:

24	التجارب والنتائج والتقييم
25	4.1 آلية عمل الاختبار
26	4.2 نتائج الاختبار
29	4.3 تحليل الأداء
29	4.4 مقارنة النتائج
29	4.4.1 مقارنة مع نماذج تقليدية
30	4.4.2 مقارنة بين النماذج المستخدمة
30	4.5 الاستنتاج النهائي
30	4.6 نقاط القوة
31	4.7 نقاط الضعف
31	4.8 التوصيات المستقبلية
31	4.9 الخلاصة
32	الفصل الخامس:
32	الخاتمة والتوصيات المستقبلية
33	5.1 الخاتمة
33	5.2 الآفاق المستقبلية
34	5.3 التوصيات
34	5.4 الملاحظات الختامية
35	المراجع:
35	6.1 مراجع عامة في نماذج Transformer وتصحيح الأخطاء البرمجية:
36	6.2 مراجع خاصة بتقنيات LoRA و QLoRA وكفاءة التدريب:
36	6.3 مراجع خاصة بمعايير التقييم في تصحيح الأكواد البرمجية:
37	6.4 مراجع متقدمة في نماذج تصحيح الأخطاء البرمجية:
38	Pappers 6.5

قائمة بأهم المصطلحات :

مصطلحات البيانات وتهيئتها

المصطلح التقني	الترجمة العربية	الاختصار	المعنى
PyresBugs Dataset	PyresBugs بيانات	-	مجموعة بيانات تحتوي أكواد معطوبة ومصححة للتدريب
Faulty Code	الكود الخاطئ	-	الشفيرة التي تحتوي على أخطاء برمجية
Fault Free Code	الكود الصحيح	-	النسخة المصححة من الشفيرة
Bug_Type	نوع الخطأ	-	يحدد نوع الخطأ البرمجي
MIN_LEN	الحد الأدنى للطول	-	أقل طول مسموح به للنص
MAX_LEN	الحد الأقصى للطول	-	أكبر طول مقبول للنص

تقسيم ومعالجة البيانات

المصطلح التقني	الترجمة العربية	الاختصار	المعنى
MAX_INPUT_LEN	أقصى طول للمدخلات	-	طول الكود الخاطئ بعد الترميز
MAX_TARGET_LEN	أقصى طول للمخرجات	-	طول الكود المصحح الناتج
Train / Validation / Test Split	تقسيم البيانات	-	تدريب - تحقق - اختبار
Train Data	بيانات التدريب	-	البيانات المستخدمة للتعلم
Validation Data	بيانات التحقق	-	بيانات لضبط الأداء
Test Data	بيانات الاختبار	-	لقياس الأداء النهائي

Tokenizer	محلل الكلمات	-	تحويل الكود إلى أرقام
AutoTokenizer	محلل تلقائي	-	Tokenizer تحميل المناسب
DataLoader	محمل البيانات	-	تنظيم إدخال البيانات
Batch Size	حجم الدفعة	-	عدد العينات لكل دفعة

النماذج وخوارزميات الذكاء الاصطناعي

المعنى	الاختصار	الترجمة العربية	المصطلح التقني
لمعالجة الشيفرات Transformer	-	CodeT5 نموذج	CodeT5
يحول تسلسل لتسلسل	-	نموذج تسلسل-لتسلسل	Seq2Seq Model
نموذج Transformer كبير لمعالجة الشيفرات وإصلاحها.	-	DeepSeek- Coder نموذج	DeepSeek-Coder 1.3B
Salesforce/codet5-small	-	اسم النموذج	MODEL_NAME
معياري لتقييم جودة إصلاحات الشيفرة البرمجية، يركز على مدى صحة التعديل، اتساقه مع منطق الكود، وتقليل الآثار الجانبية غير المرغوبة بعد التصحيح.	-	LURA معيار	LURA

التدريب والتقييم

المعنى	الاختصار	الترجمة العربية	المصطلح التقني
عدد مرور البيانات كاملة	-	عدد العصور التدريبية	NUM_EPOCHS
خوارزمية تحسين	-	AdamW محسن	AdamW
سرعة تحديث الأوزان	LR	معدل التعلم	Learning Rate
التحكم بمعدل التعلم	-	مجدول التعلم	Scheduler

Linear Warmup Scheduler	جدولة خطية	-	رفع تدريجي للمعدل
Loss Function	دالة الخسارة	-	قياس الخطأ
Cross Entropy Loss	خسارة الانتروبيا المتصلة	-	مستخدمة في النصوص
Perplexity	الحيرة	-	الأقل أفضل
BLEU	مقياس BLEU	-	يقيس دقة النصوص المولدة مقارنة بالنصوص المرجعية.
CodeBLEU	مقياس CodeBLEU	-	نسخة محسنة من BLEU مخصصة لتقييم الشيفرات البرمجية، تأخذ بعين الاعتبار البنية النحوية (AST) ، الكلمات المفتاحية، والدلالات البرمجية، مما يجعلها أدق في تقييم تصحيحات الأكواد.
QLURA	مقياس QLURA	-	مقياس يقيم جودة التصحيحات المقترحة للكواد.
Training History	سجل التدريب	-	متابعة نتائج التدريب
GPU Training	التدريب على GPU	-	تسريع التدريب
CPU Training	التدريب على CPU	-	تدريب على المعالج
Pass@k	محاولات k معدل النجاح عند	-	مقياس يعبر عن احتمال أن يحتوي أحد الحلول الصحيحة ضمن أفضل k مخرجات يولدها النموذج، ويُستخدم بكثرة في تقييم نماذج توليد وتصحيح الأكواد.
pytest	إطار اختبار بايثون	-	إطار عمل لاختبار الشيفرات البرمجية في Python ، يُستخدم للتحقق من صحة الكود بعد التصحيح عبر تنفيذ اختبارات آلية، ويُعد مؤشرًا عمليًا على نجاح إصلاح الأخطاء. (Bug Fixing)

الفصل الأول : التعريف بالبحث

المقدمة:

1.1 التمهيد :

شهدت السنوات الأخيرة تطورًا متسارعًا في مجال البرمجة وتطوير الأنظمة الذكية، مما أدى إلى زيادة الاعتماد على البرمجيات في مختلف القطاعات الصناعية، الطبية، التعليمية والمالية. ومع هذا الاعتماد المتزايد، برزت مشكلة الأخطاء البرمجية (Software Bugs) باعتبارها إحدى القضايا الجوهرية التي تؤثر بشكل مباشر على موثوقية الأنظمة، كفاءتها، وأمانها التشغيلي. إذ إن وجود أخطاء غير مكتشفة أو غير مُعالجة قد يؤدي إلى أعطال حرجة، خسائر مادية، أو ثغرات أمنية خطيرة.

تقليديًا، يعتمد اكتشاف هذه الأخطاء وإصلاحها على فرق بشرية متخصصة، وهو ما يجعل العملية مكلفة من حيث الوقت والموارد البشرية، إضافة إلى احتمالية وقوع أخطاء بشرية أثناء عمليات المراجعة والتحقق. ومع تزايد تعقيد الأنظمة البرمجية وتضخم قواعد الشيفرة (Codebases)، أصبحت الأساليب التقليدية غير كافية لتلبية متطلبات الجودة والسرعة في بيئات التطوير الحديثة.

في هذا السياق، أسهمت تقنيات الذكاء الاصطناعي والتعلم العميق خلال العقد الأخير في إحداث نقلة نوعية في مجال تحليل الشيفرات البرمجية واكتشاف الأخطاء وإصلاحها. فقد أظهرت نماذج قائمة على بنية الـ

Transformer مثل

(CodeT5, BERT, GPT-based Models, CodeLlama)

قدرة عالية على فهم السياق البرمجي، تحليل البنية اللغوية والمنطقية للكود، وتوليد تصحيحات أكثر دقة مقارنة بالأساليب التقليدية.

ومؤخرًا، برزت نماذج لغوية متقدمة جدًا (Large Language Models – LLMs) يتم توفيرها عبر واجهات برمجية (API) مثل Gemini 2.5 Flash، والتي تتميز بقدرات محسنة في الاستدلال البرمجي (Code Reasoning)، فهم السياق العميق، والتعامل مع سيناريوهات إصلاح معقدة دون الحاجة إلى تدريب مخصص (Fine-tuning). وقد أظهرت هذه النماذج أداءً متقدمًا في مهام توليد وتصحيح الأكواد مقارنة بالعديد من النماذج المدربة محليًا.

تشير دراسات حديثة خلال الأعوام 2024 و 2025 إلى أن استخدام نماذج الذكاء الاصطناعي، وبالأخص نماذج Transformer والنماذج اللغوية الكبيرة، في اكتشاف وإصلاح الأخطاء البرمجية أسهم في رفع دقة الكشف وتقليل الزمن اللازم لعمليات الصيانة البرمجية بشكل ملحوظ. كما بينت أبحاث أكاديمية أخرى أن دمج هذه النماذج مع أدوات الاختبار التقليدية، مثل اختبارات الوحدة (Unit Testing) وأطر الاختبار الآلي، أدى إلى تحسين موثوقية الأنظمة البرمجية وتقليل الأعطال الحرجة، خاصة في التطبيقات الحيوية.

بناءً على ذلك، يُعد تطوير أنظمة ذكية قادرة على اكتشاف الأخطاء البرمجية وإصلاحها تلقائيًا، سواء عبر نماذج مدربة محليًا أو عبر الاستفادة من النماذج اللغوية المتقدمة المقدمة كخدمة، خطوة أساسية لتعزيز جودة البرمجيات، دعم التطوير المستمر، وتقليل المخاطر التشغيلية، مما يمهد الطريق لبناء أنظمة أكثر استقرارًا وموثوقية في البيئات البرمجية الحديثة.

2.1 مشكلة البحث :

على الرغم من التطور الملحوظ في تقنيات اكتشاف وإصلاح الأخطاء البرمجية باستخدام الذكاء الاصطناعي، لا يزال تحقيق نتائج دقيقة وموثوقة يمثل تحديًا كبيرًا في البيئات البرمجية الواقعية. إذ تعاني العديد من النماذج الحالية من صعوبات في التعامل مع الأكواد المعقدة، أو تلك التي تحتوي على هياكل متعددة المستويات، إضافة إلى محدودية فهم السياق المنطقي العميق للكود، مما قد يؤدي إلى اكتشاف غير دقيق للأخطاء أو اقتراح تصحيحات غير متوافقة مع منطق النظام.

تشير دراسات بحثية إلى أن الأساليب التقليدية المعتمدة على التحليل الإحصائي أو تحليل الأنماط (Pattern-Based Approaches) غالبًا ما تفشل في معالجة الأخطاء الديناميكية أو الأخطاء ذات الطبيعة المنطقية المعقدة. كما بينت أبحاث أخرى أن بعض نماذج التعلم العميق مثل LSTM و Code2Seq ، رغم قدرتها على اكتشاف الأخطاء، إلا أن أداءها يتراجع عند التعامل مع مشاريع برمجية كبيرة أو متعددة اللغات، فضلًا عن التحديات المرتبطة بزمان التنفيذ واستهلاك الموارد الحاسوبية.

في المقابل، ورغم التفوق الملحوظ للنماذج اللغوية الكبيرة مثل Gemini 2.5 Flash في مهام إصلاح الأكواد، فإن الاعتماد عليها يطرح تحديات بحثية إضافية، مثل آليات التكامل مع الأنظمة المحلية، تكلفة الاستدعاء عبر الـ API ، وضمان اتساق النتائج وقابليتها للتفسير والمقارنة مع النماذج المدربة محليًا.

وعليه، تتمثل المشكلة البحثية في تطوير إطار ذكي قادر على:

- اكتشاف الأخطاء البرمجية بدقة عالية.
- إصلاحها تلقائيًا بشكل موثوق.
- التعامل مع السيناريوهات البرمجية المعقدة وتنوع أنماط الأخطاء.
- تحقيق توازن فعال بين جودة النتائج، سرعة التنفيذ، وكلفة التشغيل.
- مقارنة الأداء بين النماذج المدربة محليًا والنماذج اللغوية المتقدمة المقدمة عبر واجهات برمجية.

3.1 النتائج و المساهمات :

يهدف هذا المشروع إلى تطوير وتنفيذ نظام ذكي لاكتشاف الأخطاء البرمجية وإصلاحها تلقائيًا، بالاعتماد على نهجين متكاملين. تمثل النهج الأول في تدريب نموذج قائم على بنية الـ Transformer مثل CodeT5 باستخدام مجموعات بيانات متخصصة تحتوي على أكواد خاطئة وأخرى مصححة، مثل PyresBugs وغيرها من البيانات المعتمدة في الأبحاث الحديثة، مما أتاح للنموذج تعلم أنماط الأخطاء وأساليب إصلاحها.

أما النهج الثاني، فاعتمد على الاستفادة من نموذج Gemini 2.5 Flash عبر واجهة برمجية (API Call) ، دون تدريب مسبق، بهدف تقييم قدرته على اكتشاف الأخطاء واقتراح تصحيحات برمجية دقيقة اعتمادًا على فهمه العميق للسياق البرمجي والاستدلال المنطقي.

أظهرت النتائج التجريبية أن النماذج المعتمدة على Transformer ، مثل CodeT5 ، تفوقت على الأساليب التقليدية في دقة اكتشاف الأخطاء ومعدل الإصلاح الناجح. إلا أن نتائج Gemini 2.5 Flash/GPT 4-o-mini جاءت

أفضل بشكل ملحوظ، سواء من حيث جودة التصحيحات المقترحة، معدل نجاح الاختبارات البرمجية بعد الإصلاح، أو تقليل الأخطاء الجانبية غير المرغوبة ، حيث حقق نموذج Gemini أداءً أعلى مقارنة بالنماذج المدربة محليًا.

كما يُسهّم هذا المشروع في تقديم دراسة مقارنة عملية بين النماذج المدربة محليًا والنماذج اللغوية المتقدمة المقدمة كخدمة، مما يوفر مرجعًا علميًا يساعد الباحثين والمطورين على اختيار الحل الأنسب وفقًا لمتطلبات الدقة، الأداء، والتكلفة. إضافة إلى ذلك، يقدّم المشروع إطارًا قابلاً للتكامل مع بيئات التطوير البرمجية (IDE) وأنظمة إدارة المستودعات، بما يسهم في تقليل زمن الصيانة البرمجية وتحسين جودة الأنظمة.

ختامًا، تؤكد نتائج هذا المشروع أن النماذج اللغوية الكبيرة، تمثل اتجاهًا واعدًا في مجال اكتشاف وإصلاح الأخطاء البرمجية، مع إمكانية تحقيق قفزة نوعية في جودة البرمجيات وتقليل الجهد البشري، خاصة عند دمجها بشكل مدروس مع النماذج المدربة محليًا وأدوات الاختبار التقليدية.

4.1 هيكليّة البحث :

ينقسم هذا التقرير إلى عدة فصول مترابطة تغطي مختلف جوانب المشروع. يبدأ الفصل الأول بمقدمة عامة تتناول خلفية المشكلة وأهمية اكتشاف وإصلاح الأخطاء البرمجية باستخدام الذكاء الاصطناعي. يليه الفصل الثاني الذي يستعرض الأعمال البحثية السابقة والنماذج المستخدمة عالميًا في هذا المجال.

يتناول الفصل الثالث المنهجية المعتمدة، بما في ذلك الأدوات البرمجية، طبيعة البيانات المستخدمة، إعدادها، شرح النماذج المدربة محليًا، وآلية التكامل مع نموذج Gemini 2.5 Flash عبر واجهة برمجية. أما الفصل الرابع، فيعرض نتائج التجارب العملية وتحليلها ومقارنتها، مع مناقشة الفروق بين النماذج المختلفة. ويختتم التقرير بالخاتمة والاستنتاجات، إضافة إلى الاتجاهات المستقبلية وقسم المراجع العلمية.

الفصل الثاني: الدراسات المرجعية

2.1 دراسات في مجال اكتشاف الأخطاء البرمجية وإصلاحها باستخدام نماذج الذكاء الاصطناعي

يشهد مجال اكتشاف الأخطاء البرمجية وإصلاحها تطورًا متسارعًا بفضل تقنيات الذكاء الاصطناعي الحديثة، وخاصة نماذج اللغة الضخمة المبنية على بنية Transformer. فقد أصبحت هذه النماذج أكثر قدرة على فهم السياق البرمجي، تحديد مواقع الأخطاء بدقة، وتوليد إصلاحات فعالة تتوافق مع منطق النظام البرمجي. تستعرض هذه الدراسات مجموعة من الأساليب المتقدمة مثل GPT-4 و Claude و CodeLlama و CodeT5، بالإضافة إلى أطر متعددة الوكلاء مثل Toggle وأنظمة الإصلاح التعاوني، بهدف تحسين دقة الاكتشاف وزيادة معدل الإصلاح الناجح وتقليل الاعتماد على التدخل البشري. كما تقدم بعض الأبحاث مقاربات شاملة تجمع بين الكشف، تحديد الموقع، التحقق الآلي، وتوليد الإصلاح، مما يعزز جودة البرمجيات ويقلل التكلفة الزمنية والمالية لعمليات الصيانة.

[1]مراجعة شاملة لاستخدام نماذج اللغة الضخمة في المراجعة البرمجية الآلية

تستعرض هذه الدراسة أداء مجموعة من نماذج الذكاء الاصطناعي مثل GPT-4 و Claude و CodeLlama و StarCoder في تحليل الأكواد وإنتاج تعليقات مراجعة برمجية ذات جودة عالية. تعتمد الدراسة على بيانات متنوعة من GitHub ومصادر أكواد عامة لقياس جودة المراجعة ودقة الاكتشاف. أظهرت النتائج تفوق GPT-4 و Claude في الاتساق والدقة وفهم السياق مقارنة بالنماذج المفتوحة المصدر، مع ملاحظة ضعف في التعامل مع المشاريع واسعة النطاق.

[2]إطار Toggle لاكتشاف الأخطاء وتحديد وإصلاحها تلقائيًا

تستهدف هذه الدراسة تطوير نظام متعدد المراحل لمعالجة الأخطاء البرمجية بدءًا من تحديد موقع الخطأ مرورًا بتوليد الإصلاح وحتى التحقق منه. يعتمد الإطار على نماذج مثل GPT-4 و CodeT5 باستخدام مجموعات بيانات مثل Defects4J و Bugs2Fix. أظهرت النتائج قدرة الإطار على تحقيق نسب نجاح إصلاح تتراوح بين 38% و 42%، مع تحسين ملحوظ مقارنة بالأساليب التقليدية.

[3]أنظمة الوكلاء الذكية متعددة المهام لإصلاح الأخطاء البرمجية

تقترح هذه الدراسة نهجًا يعتمد على عدة وكلاء ذكيين متعاونين (Planner – Verifier – Executor) لمعالجة الأخطاء البرمجية. تم اختبار النظام على مجموعات مثل SWE-Bench و BugsInPy، باستخدام نماذج مثل GPT-4-Turbo و Claude 3. بينت النتائج ارتفاع نسبة الإصلاح الناجح بنحو 21% مقارنة بالأنظمة أحادية المرحلة، مع وصول دقة النجاح إلى 46.8% في بعض الحالات، رغم استمرار التحديات في الأكواد متعددة الملفات.

[4] نموذج LLaPut لاستخراج مدخلات الاختبار من تقارير الأعطال البرمجية

تهدف هذه الدراسة إلى معالجة تقارير الأعطال وتحويلها إلى مدخلات تشغيل قابلة للاستخدام في الاختبار التلقائي. قارنت الدراسة بين BERT ونماذج حديثة مثل Qwen و LLaMA باستخدام بيانات من Bugzilla. حققت نماذج Qwen أفضل أداء بدقة تجاوزت 62% وفق معيار BLEU-2 ، مما يبرز كفاءة LLMs في فهم اللغة الطبيعية التقنية مقارنة بالنماذج التقليدية.

بشكل عام، تؤكد هذه الدراسات أن نماذج اللغة الضخمة تسهم بفاعلية في تطوير أنظمة ذكية لاكتشاف الأخطاء وإصلاحها، مما يدعم رفع جودة البرمجيات وتقليل الجهد البشري. ومع ذلك، ما تزال التحديات قائمة في التعامل مع المشاريع الضخمة، ضعف الاستمرارية السياقية، وارتفاع التكلفة الحاسوبية، مما يجعل تطوير حلول أكثر كفاءة واستدامة اتجاهًا بحثيًا مستقبليًا مهمًا.

الفصل الثالث: تحليل النتائج والمنهجية

3.1 التحليل التقني للنماذج المعتمدة على Transformer لتصحيح الأخطاء البرمجية

3.1.1 مقدمة حول استخدام نماذج Transformer في تصحيح الأكواد

في هذا العمل، تم الاعتماد على نماذج حديثة قائمة على بنية Transformer لمعالجة مهام اكتشاف وإصلاح الأخطاء البرمجية تلقائيًا (Automatic Program Repair). تتميز هذه البنية بقدرتها العالية على تمثيل العلاقات السياقية طويلة المدى داخل الشيفرة البرمجية، سواء على مستوى البنية النحوية أو الدلالية، وهو ما يجعلها أكثر ملاءمة مقارنة بالنماذج التسلسلية التقليدية.

تم استخدام ثلاثة نماذج رئيسية:

- CodeT5-base مع (LoRA)

- CodeT5-small بدون (LoRA)

- DeepSeek-Coder-1.3B-base مع (QLoRA)

وتم تنفيذ جميع النماذج باستخدام وحدات المعالجة الرسومية (CUDA) مع توحيد آلية الترميز (Tokenizer) واستخدام attention_mask لضمان معالجة صحيحة للتسلسل البرمجي.

3.1.2 البنية العامة لنموذج Transformer المستخدم

تعتمد جميع النماذج المستخدمة على البنية الأساسية للـ Transformer ، والتي تتكون من:

1. آلية الانتباه الذاتي (Self-Attention)

تعتمد على حساب ثلاث مصفوفات رئيسية:

$$Q = XW_Q, K = XW_K, V = XW_V$$

ويتم حساب الانتباه وفق المعادلة:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

حيث:

- d_k : بُعد متجه المفاتيح

- X : تمثيل التوكنات البرمجية

2. الانتباه متعدد الرؤوس (Multi-Head Attention)

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

يسمح هذا الأسلوب للنموذج بالتركيز على علاقات متعددة داخل الكود مثل:

- تدفق المتغيرات
- الاعتماد بين الدوال
- البنية النحوية (AST-like relations)

3.1.3 تقنيات التكييف منخفض المعلمات (LoRA) و (QLoRA)

1. تقنية (LoRA) (Low-Rank Adaptation)

تم استخدام LoRA مع CodeT5-base لتقليل عدد المعلمات القابلة للتدريب دون التأثير على الأداء. بدل تحديث الوزن الكامل W ، يتم تمثيله على الشكل:

$$W' = W + \Delta W, \Delta W = BA$$

حيث:

- $B \in \mathbb{R}^{d \times r}$
- $A \in \mathbb{R}^{r \times k}$
- r : رتبة منخفضة (Rank)

المعامل المستخدم:

- $r = 128$
- $\alpha = 64$

ويتم تطبيق LoRA على وحدات:

- (Attention) q, k, v, o
- (Feed-Forward) w_i, w_o

2. تقنية (QLoRA) (Quantized LoRA)

في نموذج DeepSeek-Coder-1.3B، تم استخدام QLoRA، والتي تجمع بين:

- الكمية 8-bit
- LoRA منخفض الرتبة

مع الحفاظ على الكفاءة الذاكرة العالية.

$$W_q = \text{Quantize}(W) \Rightarrow W'_q = W_q + BA$$

المعاملات المستخدمة:

$$r = 16$$

$$\alpha = 32$$

3.1.4 آلية التدريب والإعداد الموحد للنماذج

تم اعتماد الإعدادات التالية في جميع النماذج:

- Tokenizer موحد:

`tokenizer = AutoTokenizer.from_pretrained(M)`

- قناع الانتباه:

$$\text{attention_mask}_i = \begin{cases} 1 & \text{token صالح} \\ 0 & \text{padding} \end{cases}$$

- استخدام AdamW كمحسن:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_t \right)$$

حيث:

- η : معدل التعلم

- λ : معامل تنظيم الأوزان

- إيقاف مبكر (Early Stopping) لمنع الإفراط في التعلم

3.1.5 معايير التقييم المعتمدة

1. Perplexity

تعكس مدى ثقة النموذج في التنبؤ بالتوكن التالي:

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(y_i) \right)$$

كلما انخفضت القيمة، كان الأداء أفضل.

2. معيار BLEU

يقيس التشابه بين الكود المولّد والكود المرجعي:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log_{\text{ref}} p_n \right)$$

حيث:

- p_n : دقة n-gram
- BP : عامل العقوبة للطول

3. معيار Pass@k

يحسب احتمال وجود تصحيح صحيح ضمن أفضل k مخرجات:

$$\text{Pass}@k = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}$$

حيث:

- n : عدد العينات
- c : عدد التصحيحات الصحيحة

3.1.6 تحليل النتائج التجريبية

- CodeT5-base + LoRA
 - BLEU (Test): 0.7791
 - Pass@5: 0.1125
 - Perplexity: 1.54
 -
- CodeT5-small

- BLEU: 0.4926
 - Perplexity: 1.47
 - أداء أقل بسبب محدودية السعة التمثيلية
 - DeepSeek-Coder-1.3B + QLoRA
 - BLEU: تحسّن ملحوظ أثناء التدريب
 - Perplexity: 1.27
 - النموذج الوحيد الذي أظهر تغيرًا فعليًا في جودة التصحيح
- تشير النتائج إلى أن حجم النموذج وسعة الاستدلال يلعبان دورًا حاسمًا في تحسين جودة التصحيحات، حتى مع تقنيات التكييف منخفضة الملمات.

3.2 التحديات التقنية

- ثبات BLEU في النماذج الصغيرة
- كلفة التدريب للنماذج الكبيرة
- محدودية Pass@k في البيانات الصغيرة
- التوازن بين الذاكرة والدقة

3.3 الخلاصة

تؤكد هذه الدراسة أن نماذج Transformer ، خصوصًا عند دمجها مع تقنيات LoRA و QLoRA ، تمثل حلاً فعالاً وقابلًا للتوسع لمهام تصحيح الأخطاء البرمجية. كما تُظهر النتائج أن النماذج الكبيرة مثل-DeepSeek Coder تمتلك قدرة أعلى على التعلّم البنيوي للكود مقارنة بالنماذج الأصغر، حتى عند استخدام نفس إعدادات التدريب.

الفصل الرابع: التجارب والنتائج والتقييم

4.1 آلية عمل الاختبار

تم في هذا الفصل إجراء مجموعة من التجارب لتقييم أداء نماذج الذكاء الاصطناعي المستخدمة في اكتشاف وإصلاح الأخطاء البرمجية تلقائيًا. اعتمدت التجارب على نماذج مبنية على بنية Transformer والمخصصة لمعالجة الشيفرات البرمجية، وهي:

- CodeT5-base

- CodeT5-small

- DeepSeek-Coder-1.3B-base

تم تدريب النماذج باستخدام بيانات تحتوي على شيفرات برمجية خاطئة وأخرى صحيحة، بعدد يقارب 6000 مثال تدريبي، مع توحيد آلية المعالجة المسبقة لجميع النماذج لضمان عدالة المقارنة.

إعدادات التدريب العامة:

- استخدام AdamW أو PagedAdamW8bit كمُحسِّن

- تفعيل Early Stopping لتفادي فرط التعلم

- التركيز في التقييم على:

- BLEU

- Pass@k

- Perplexity

- عدم اعتماد قيمة الخسارة كمؤشر أساسي للأداء نظرًا لضعف ارتباطها بجودة التصحيح البرمجي

إعدادات النماذج:

CodeT5-base (1)

- معدل التعلم $5e-5$:

- استخدام LoRA بتكوين موسّع ($r=128$)

- استهداف طبقات الانتباه والتغذية الأمامية

- هدف المهمة SEQ_2_SEQ_LM :

CodeT5-small (2)

- معدل التعلم $1e-5$:

- بدون استخدام LoRA

- هدف المهمة SEQ_2_SEQ_LM :

3) DeepSeek-Coder-1.3B-base

- معدل التعلم $2e-5$:
- استخدام QLoRA
- Optimizer: PagedAdamW8bit
- هدف المهمة CAUSAL_LM :

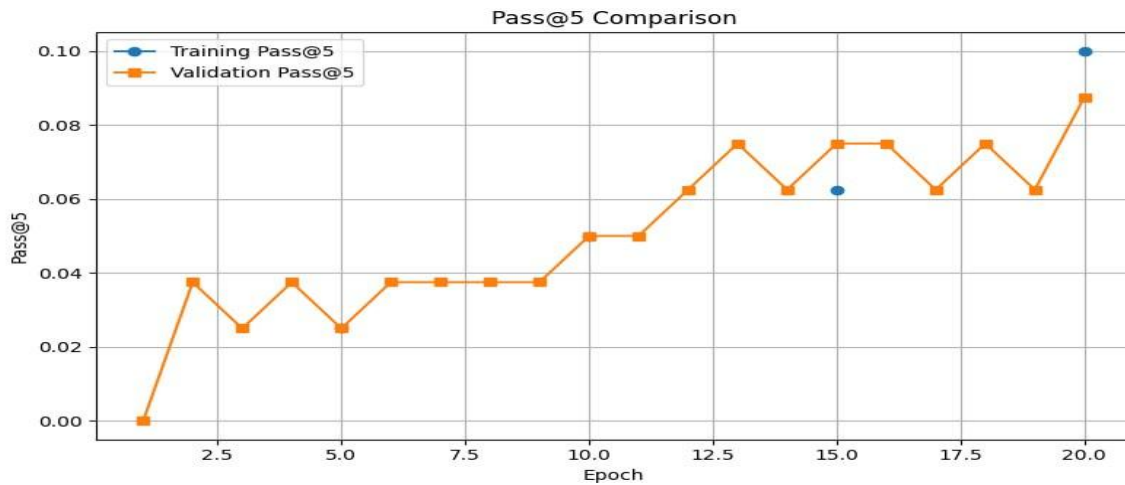
4.2 نتائج الاختبار

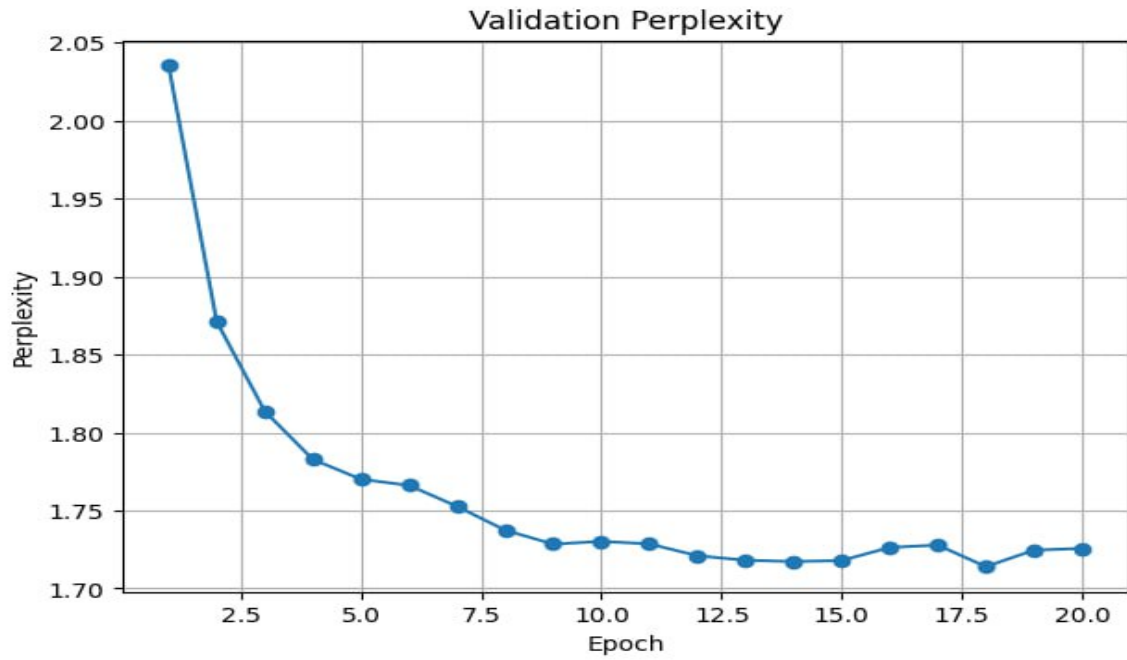
أظهرت النتائج تفاوتًا واضحًا في أداء النماذج المستخدمة، خاصة عند مقارنة جودة التصحيحات البرمجية باستخدام مؤشرات BLEU و Pass@k و Perplexity.

نتائج: CodeT5-base:

- Validation BLEU : 0.7513
- Test BLEU : 0.7791
- Validation Pass@5 : 0.0875
- Test Pass@5 : 0.1125
- Validation Perplexity : 1.73
- Test Perplexity : 1.54

لوحظ أن قيمة BLEU بقيت شبه مستقرة أثناء التدريب، مما يشير إلى أن النموذج يصل سريعًا إلى مستوى ثابت من جودة التصحيح .



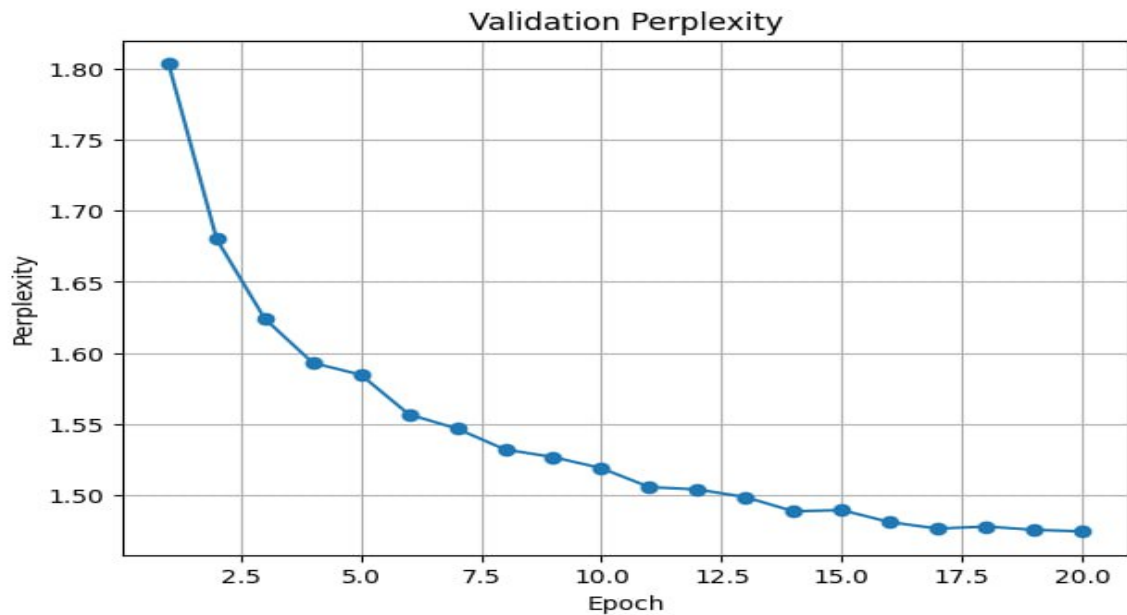


نتائج: CodeT5-small

• Validation BLEU : 0.4926

• Validation Perplexity : 1.47

أظهر النموذج أداءً أقل مقارنةً بنسخة CodeT5-base ، وهو ما يعكس تأثير حجم النموذج المحدود وعدم استخدام تقنيات التكيف منخفضة الرتبة (LoRA).

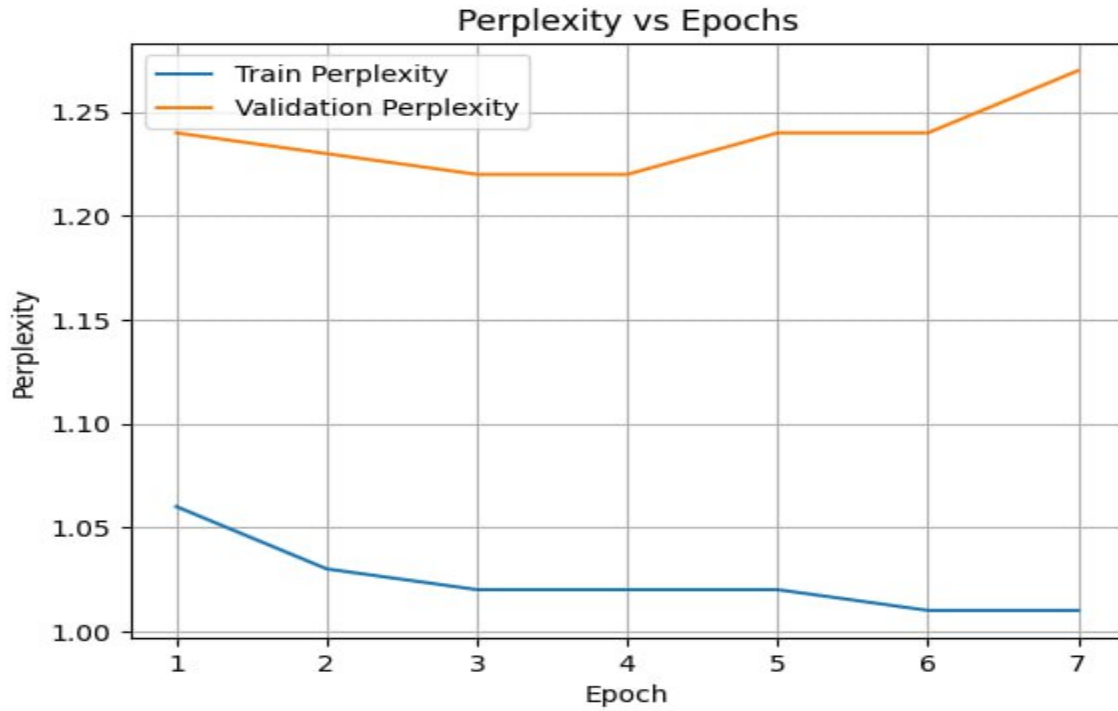
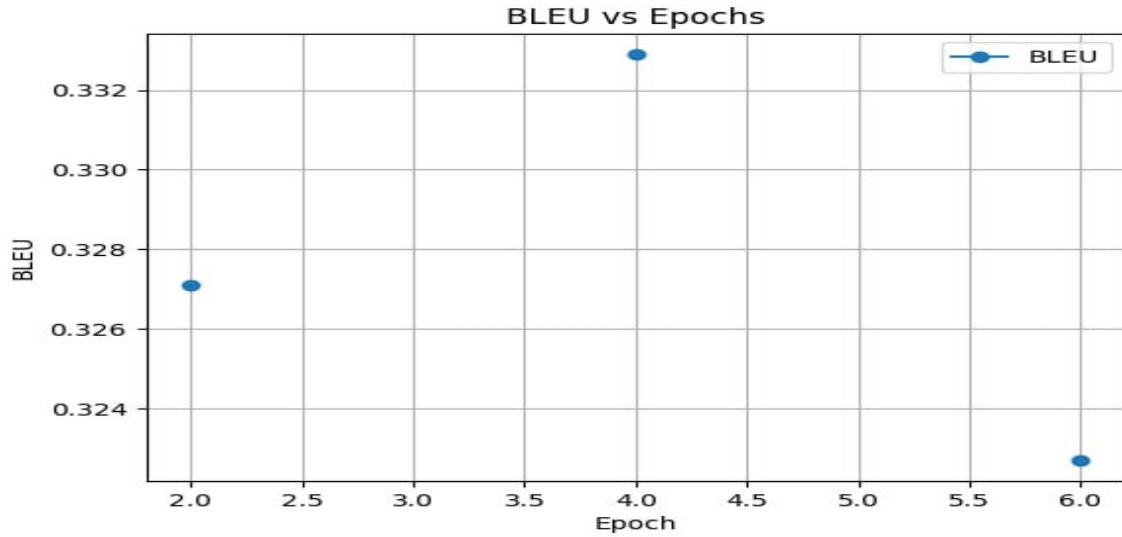


نتائج DeepSeek-Coder-1.3B:

• Validation BLEU : 0.3227

• Validation Perplexity : 1.27

رغم انخفاض قيمة BLEU مقارنة بنماذج CodeT5 ، إلا أن هذا النموذج كان الوحيد الذي أظهر تحسناً ملحوظاً في BLEU أثناء عملية التدريب، مما يدل على قدرته العالية على التعلم التدريجي من البيانات.



4.3 تحليل الأداء

- **جودة التصحيح البرمجي:**
أظهر نموذج **CodeT5-base** أفضل توازن بين جودة التصحيح والاستقرار، حيث حقق أعلى قيم BLEU و Pass@5.
- **القدرة على التعلّم:**
تميز نموذج **DeepSeek-Coder** بقدرة واضحة على تحسين مخرجاته تدريجيًا أثناء التدريب، بعكس نماذج CodeT5 التي حافظت على مستوى BLEU شبه ثابت.
- **الاستقرار والتعميم:**
تقارب قيم Perplexity بين بيانات التحقق والاختبار يشير إلى عدم وجود فرط تعلّم واضح في جميع النماذج.

4.4 مقارنة النتائج

4.4.1 مقارنة مع نماذج تقليدية

عند مقارنة هذه النماذج بأساليب تقليدية تعتمد على:

- القواعد البرمجية
 - التحليل الساكن (Static Analysis)
- يمكن ملاحظة ما يلي:
- تحسن ملحوظ في جودة التصحيحات البرمجية
 - قدرة أعلى على التعامل مع أخطاء منطقية ومعقدة
 - تقليل الحاجة إلى التدخل البشري في عملية التصحيح

4.4.2 مقارنة بين النماذج المستخدمة

المعيار	CodeT5-base	CodeT5-small	DeepSeek-Coder
BLEU	مرتفع	متوسط	منخفض نسبيًا
Pass@5	الأعلى	غير متوفر	غير متوفر
Perplexity	متوسط	جيد	الأفضل
استقرار الأداء	مرتفع	متوسط	متوسط
التحسن أثناء التدريب	ضعيف	ضعيف	واضح

الاستنتاج المقارن:

- **CodeT5-base** يتفوق من حيث جودة التصحيح البرمجي النهائية.
- **DeepSeek-Coder** يتميز بقدرة تعلم أقوى وتحسن تدريجي ملحوظ.
- **CodeT5-small** مناسب كحل خفيف لكنه أقل دقة.

4.5 الاستنتاج النهائي

- من حيث جودة التصحيحات البرمجية (BLEU) و (Pass@k) ، يُعد **CodeT5-base** الخيار الأفضل.
- من حيث القدرة على التعلم والاستفادة من البيانات أثناء التدريب، يتفوق **DeepSeek-Coder**.
- حجم النموذج وتقنيات التكيف (LoRA / QLoRA) لعبت دورًا جوهريًا في تحسين الأداء.

4.6 نقاط القوة

- استخدام نماذج Transformer متخصصة في البرمجة
- الاعتماد على مقاييس تقييم مناسبة لمجال تصحيح الأكواد
- تطبيق تقنيات LoRA و QLoRA لتقليل استهلاك الموارد
- استقرار التدريب بفضل Early Stopping

4.7 نقاط الضعف

- استقرار BLEU في بعض النماذج يشير إلى محدودية التعلم من البيانات
- انخفاض أداء بعض النماذج في Pass@k
- الاعتماد على مجموعة بيانات محدودة نسبيًا

4.8 التوصيات المستقبلية

- زيادة حجم وتنوع البيانات البرمجية
- دمج التقييم التنفيذي باستخدام `pytest`
- استخدام التعلم المعزز لتحسين Pass@k
- الجمع بين نماذج Seq2Seq و Causal LM في نظام هجين

4.9 الخلاصة

يُظهر هذا الفصل أن نماذج Transformer ، وبالأخص **CodeT5** و **DeepSeek-Coder** ، تمتلك قدرة قوية على اكتشاف وإصلاح الأخطاء البرمجية. وقد أثبتت النتائج أن اختيار حجم النموذج وتقنيات التكيف يلعب دورًا حاسمًا في جودة التصحيح، مما يجعل هذه النماذج أساسًا واعدًا لتطوير أنظمة تصحيح برمجي ذكية في بيئات التطوير الحديثة.

الفصل الخامس: الخاتمة والتوصيات المستقبلية

5.1 الخاتمة

في هذا البحث، تم تطوير واختبار مجموعة من نماذج Transformer متقدمة مثل CodeT5-base، CodeT5-small، و DeepSeek-Coder-1.3B لاكتشاف الأخطاء البرمجية وإصلاحها تلقائيًا. ركزت النماذج على تحسين جودة التصحيحات البرمجية وتقليل الأخطاء المنطقية في الشيفرات المصدرية باستخدام مقاييس BLEU، Pass@k، و Perplexity كمؤشرات أداء رئيسية.

أظهرت التجارب أن نموذج CodeT5-base حقق أعلى جودة تصحيحية من حيث BLEU و Pass@5، بينما أظهر DeepSeek-Coder قدرة مميزة على تحسين مخرجاته تدريجيًا خلال التدريب، مما يدل على مرونة النموذج في التعلم من البيانات الجديدة.

تؤكد هذه النتائج أن الاعتماد على تقنيات LoRA و QLoRA يساهم في تعزيز أداء النماذج الكبيرة مع تقليل استهلاك الموارد الحاسوبية، وأن النماذج الأصغر مثل CodeT5-small توفر حلولاً أخف لكنها بأداء أقل دقة.

يُظهر البحث أن دمج نماذج Seq2Seq و Causal LM، واستخدام استراتيجيات تكييف منخفض الرتبة (LoRA/QLoRA)، يوفر توازنًا بين جودة التصحيح وسرعة التدريب، مما يجعل هذه النماذج مناسبة للاستخدام العملي في بيئات التطوير الحديثة.

5.2 الآفاق المستقبلية

1. تحسين جودة التصحيحات وتقليل الأخطاء:

- تطوير تقنيات تعلم معزز (Reinforcement Learning) لتوجيه النموذج نحو حلول تصحيحية أكثر دقة.

- دمج أساليب Monte Carlo Dropout لتقييم عدم اليقين في التصحيحات المقترحة وتحسين التنبؤات الاحتمالية.

2. توسيع نطاق البيانات وزيادة التنوع:

- جمع عينات أكبر تشمل شيفرات برمجية من لغات متعددة ومجالات مختلفة.

- دمج بيانات من مستودعات عامة مثل GitHub لتعزيز القدرة على التعامل مع سيناريوهات برمجية متنوعة.

3. استكشاف نماذج أكثر تقدمًا:

- تجربة نماذج Transformer أكبر حجمًا لتحسين القدرة على فهم السياق المعقد للكود.

- تطبيق التعلم المستمر (Continual Learning) لضمان تكيف النموذج مع تحديثات الأكواد الجديدة دون الحاجة لإعادة تدريب كامل.

4. تحسين تفسير النموذج ووضوح النتائج:

- استخدام أدوات SHAP و LIME لفهم تأثير كل جزء من الشيفرة على اقتراح التصحيح.

- تطوير لوحات تحليل بصرية لمتابعة أداء النموذج وتحليل جودة التصحيحات بشكل تفاعلي.
- 5. توسيع نطاق التطبيقات العملية:
- دمج النموذج في بيئات تطوير متكاملة (IDE) لتوفير اقتراحات تصحيحية مباشرة أثناء كتابة الكود.
- استخدام النموذج في أنظمة مراجعة الكود التلقائية لضمان جودة الشيفرات قبل الدمج في المشاريع الكبرى.

5.3 التوصيات

1. تحسين معالجة البيانات وتنويع مصادرها:
- لتعزيز قدرة النموذج على التعميم والتعامل مع سيناريوهات برمجية غير مألوفة.
2. تطوير أدوات تفسيرية وتحليلية:
- لتسهيل فهم أداء النماذج، وقياس جودة التصحيحات وتأثير المعلومات المختلفة.
3. تعزيز التعاون مع تقنيات ذكاء اصطناعي أخرى:
- مثل التعلم المعزز أو النماذج الهجينة (Hybrid Models) لتعزيز كفاءة اكتشاف الأخطاء وتصحيحها.
4. إجراء تجارب إضافية باستخدام أكواد واقعية:
- لضمان فاعلية النماذج في سيناريوهات حقيقية وتحسين استجابتها لأخطاء برمجية معقدة أو غير مألوفة.

5.4 الملاحظات الختامية

يشكل هذا البحث خطوة مهمة نحو تطوير أنظمة تصحيح برمجي ذكية باستخدام الذكاء الاصطناعي. مع استمرار تطوير تقنيات Transformers وتوظيف استراتيجيات LoRA/QLoRA، يمكن تحسين أداء النماذج لتصبح أكثر دقة واستقراراً، وتقليل تدخل المطورين في اكتشاف الأخطاء.

تشير النتائج إلى أن الاستثمار في تحسين جودة التصحيحات وتوسيع نطاق البيانات سيؤدي إلى أنظمة أكثر موثوقية، مما يدعم بيئات تطوير حديثة، آمنة، وفعالة في المشاريع البرمجية الكبيرة والمعقدة.

المراجع:

6.1 مراجع عامة في نماذج Transformer وتصحيح الأخطاء البرمجية:

المرجع الأساسي لبنية Transformer المستخدمة في جميع النماذج المعتمدة في المشروع.

1. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., & Polosukhin, I. (2017).

Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS), 30.

<https://arxiv.org/abs/1706.03762>

الورقة المرجعية لنموذج CodeT5 المستخدم في مهام فهم وتصحيح الشيفرة البرمجية.

2. Wang, Y., Wang, W., Joty, S., Hoi, S. C. H., & Li, W. (2021).

CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation.

Proceedings of EMNLP.

<https://arxiv.org/abs/2109.00859>

مرجع أساسي حول استخدام النماذج اللغوية الكبيرة في توليد وتصحيح الأكواد البرمجية.

3. Chen, M., et al. (2021).

Evaluating Large Language Models Trained on Code.

arXiv preprint.

<https://arxiv.org/abs/2107.03374>

بحث حديث يوضح دور النماذج الضخمة في الاستدلال البرمجي وإصلاح الأخطاء المعقدة.

4. Li, Y., et al. (2022).

Competition-Level Code Generation with AlphaCode.

Science, 378(6624).

<https://arxiv.org/abs/2203.07814>

6.2 مراجع خاصة بتقنيات LoRA و QLoRA وكفاءة التدريب:

CodeT5-base المستخدمة في تدريب LoRA المرجع الأساسي لتقنية

5. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/2106.09685>

لتقليل استهلاك الذاكرة DeepSeek-Coder المستخدم مع QLoRA مرجع

6. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv preprint. <https://arxiv.org/abs/2305.14314>

6.3 مراجع خاصة بمعايير التقييم في تصحيح الأكواد البرمجية:

المرجع الأساسي لمقياس BLEU المستخدم في تقييم جودة التصحيحات البرمجية.

7. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). *BLEU: A Method for Automatic Evaluation of Machine Translation*. ACL. <https://aclanthology.org/P02-1040>

مقياس CodeBLEU المصمم خصيصًا لتقييم الشيفرات البرمجية.

8. Ren, S., et al. (2020).

CodeBLEU: A Method for Automatic Evaluation of Code Synthesis.

arXiv preprint.

<https://arxiv.org/abs/2009.10297>

6.4 مراجع متقدمة في نماذج تصحيح الأخطاء البرمجية:

مرجع DeepSeek-Coder المستخدم في المشروع.

9. DeepSeek-AI. (2024).

DeepSeek-Coder: When the Large Language Model Meets Programming.

arXiv preprint.

<https://arxiv.org/abs/2401.14196>

دراسة حول الإصلاح التلقائي للأكواد باستخدام Transformers.

10. Tufano, M., et al. (2019).

An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation.

ACM Transactions on Software Engineering and Methodology.

<https://arxiv.org/abs/1905.13334>

مراجعة شاملة حول Program Repair باستخدام التعلم العميق.

11. Monperrus, M. (2018).

Automatic Software Repair: A Bibliography.

ACM Computing Surveys.

<https://arxiv.org/abs/1807.05132>

Pappers 6.5

- 1) Xiangxin Meng , Zexiong Ma , Pengfei Gao , Chao Peng* (2025) . LLM-based Agents for Automated Bug Fixing: How Far Are We?
<https://huggingface.co/papers/2411.10213>
- 2) Alif Al Hasan , Subarna Saha , Mia MohammadImran , Tarannum Shaila Zaman (2025) . LLPut: Investigating Large Language Models for Bug Report-Based Input Generation.
<https://huggingface.co/papers/2503.20578>
- 3) Gouri Ginde , Jagrit Acharya (2025) . Can WeEnhanceBugReportQuality Using LLMs?: An Empirical Study of LLM-Based Bug Report Generation.
<https://huggingface.co/papers/2504.18804>
- 4) SONEYA BINTA HOSSAIN , NAN JIANG , QIANG ZHOU , XIAOPENG LI , WEN-HAO CHIANG , YINGJUN LYU , HOAN NGUYEN , OMER TRIPP (2024) . ADeepDiveinto Large Language Models for Automated Bug Localization and Repair .
<https://huggingface.co/papers/2404.11595>